A Distributed Resource Manager and Job Scheduler for Heterogeneous Lab Computing

Operating System Term Project

Team Members: 614410111 - 張宸維、614410038 - 陳弘翔

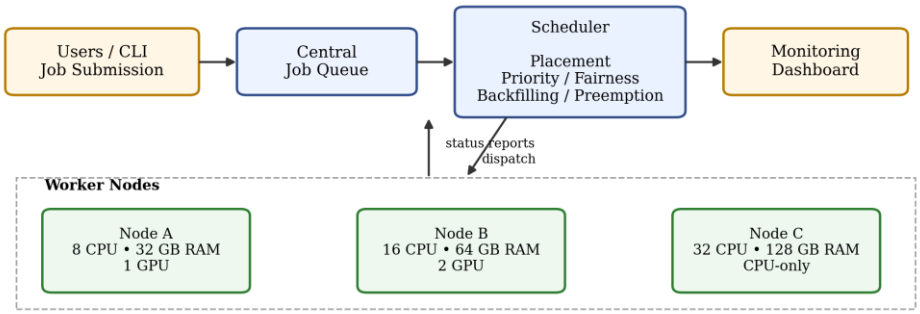
Project Overview

Modern research laboratories often contain multiple computing machines with different amounts of CPU, memory, and GPU resources. However, these resources are frequently used in an uncoordinated manner. A user may manually connect to several machines to find an available GPU, while another machine may remain mostly idle. In small laboratory environments without a dedicated cluster manager, there may be no central mechanism to decide which job should run first, where it should run, or how resources should be shared fairly. This project proposes the implementation of a distributed resource manager and job scheduler for a heterogeneous laboratory computing environment. The main goal is to treat a collection of independent machines as a unified pool of computing resources and allow users to submit jobs without manually selecting a specific machine.

System Design

The system will consist of a central scheduler and multiple worker nodes. Each worker node will periodically report its available CPU cores, memory capacity, GPU devices, and current utilization to the scheduler. A user will submit a job together with a resource request, such as the number of CPU cores, amount of memory, number or type of GPUs, and an optional priority. The scheduler will maintain a global view of the cluster and decide when and where each job should be executed. If sufficient resources are not currently available, the job will remain in a waiting queue until an appropriate allocation becomes possible. Once a job finishes, its assigned resources will be released and made available to other jobs.

Figure 1. Simplified Architecture of the Distributed Job Scheduler



Scheduling Policies

A major focus of the project will be the design and implementation of scheduling policies rather than merely forwarding jobs to idle machines. We intend to support resource-aware placement, priority, fairness, and backfilling. Placement will match each job with a node that satisfies its CPU, memory, and GPU requirements. Priority will allow important jobs to receive earlier service, while fairness will prevent one user from monopolizing the cluster. Waiting time and recent resource usage will be combined into a scheduling score. Jobs may optionally provide an estimated execution time, allowing the scheduler to backfill smaller jobs into temporarily available resources without unnecessarily delaying jobs ahead of them.

Preemption and Resource Control

We also intend to implement preemption as an advanced scheduling feature. When the cluster is full and a high-priority job is submitted, the initial implementation will preempt selected restartable low-priority jobs by terminating them and returning them to the queue, thereby releasing their allocated resources. The preempted jobs can later restart when resources become available. If time permits, we will investigate checkpoint-and-resume support so that preempted jobs can preserve execution progress rather than starting over. Each running job will also be constrained to its allocated resources so that a single task cannot consume all CPU cores or memory on a worker node.

Heterogeneous Resources and Failure Handling

The scheduler will explicitly support heterogeneous machines. For example, one node may contain a high-memory GPU, another may contain several consumer GPUs, and another may provide only CPU resources. Users will be able to request either a general resource, such as any available GPU, or a more specific resource when required. The scheduler will hide the physical location of these resources and select a suitable node automatically. The system will also monitor worker availability using periodic heartbeat messages. If a worker becomes unreachable, the scheduler will mark its resources as unavailable and report or re-queue the affected jobs.

Implementation Plan and Evaluation

We plan to implement the central scheduler as a standalone service and deploy a lightweight worker daemon on each computing node. The scheduler and workers will communicate through periodic heartbeats and job-control messages, while each worker will report its available CPU, memory, and GPU resources. The system will also provide a job submission interface and a monitoring interface that visualizes nodes, queued jobs, running jobs, and allocation decisions. CPU and memory limits will be enforced using Linux control groups (cgroups), while GPU access will be restricted to the devices assigned by the scheduler. The implementation will be evaluated on a small cluster of at least three heterogeneous physical or virtual worker nodes using workloads with different CPU, memory, GPU, estimated duration, and priority requirements. The main metrics will include resource utilization, average waiting time, job completion time, fairness among users, and responsiveness to high-priority jobs. We will compare simple first-come-first-served scheduling with our resource-aware design that includes priority and backfilling.

Expected Outcomes and Intended Accomplishments

By the end of the project, we intend to deliver a working distributed job scheduling system rather than only a simulation. The prototype should allow multiple machines to join a unified resource pool, let users submit jobs with CPU, memory, and GPU requirements, automatically place jobs on suitable machines, queue jobs when resources are unavailable, support priority and fairness, improve utilization through backfilling, detect worker failures, and recover or re-queue affected jobs. We also aim to demonstrate preemption, with checkpoint-and-resume support treated as an optional extension if the core system is completed early. We will consider the prototype successful if it can schedule concurrent jobs across at least three heterogeneous worker nodes and demonstrate, under resource contention, higher resource utilization or lower average waiting time than a simple first-come-first-served baseline. Through this implementation, the project will explore operating-system concepts including scheduling, resource allocation, isolation, process control, and fault handling. The final demonstration will show real-time scheduling under job arrivals, resource changes, priority changes, and worker failures.